

Universidade do Estado do Rio de Janeiro

Jully Moura

Atividade 6

Rio de Janeiro

2025

Exercício 01

```
1 #include <TFile.h>
2 #include <TTree.h>
3 #include <TH1F.h>
4 #include <TCanvas.h>
5 #include <TMath.h>
6 #include <TF1.h>
7 #include <TChain.h>
8 #include <TLorentzVector.h>
9
10 void Exercicio6() {
11
12     // Abrindo o arquivo ROOT
13     TChain* fChain = new TChain("Events");
14     fChain->Add("76A312CE-F9C3-5F4D-9093-F26D7C50BF61.root");
15
16     // Declarando variáveis
17     UInt_t nTau, nMuon, nElectron, nJet;
18     TLorentzVector muon1, muon2;
19     float Tau_pt[9], Muon_pt[9], Electron_pt[9], Jet_pt[9], Tau_eta[9], Muon_eta[9],
20         Electron_eta[9], Jet_eta[9], Tau_phi[9], Muon_phi[9], Electron_phi[9], Jet_phi
21         [9];
22
23     // Ligando as variáveis tree
24     fChain->SetBranchAddress("nTau", &nTau);
25     fChain->SetBranchAddress("nMuon", &nMuon);
26     fChain->SetBranchAddress("nElectron", &nElectron);
27     fChain->SetBranchAddress("nJet", &nJet);
28     fChain->SetBranchAddress("Tau_pt", &Tau_pt);
29     fChain->SetBranchAddress("Muon_pt", &Muon_pt);
30     fChain->SetBranchAddress("Electron_pt", &Electron_pt);
31     fChain->SetBranchAddress("Jet_pt", &Jet_pt);
32     fChain->SetBranchAddress("Tau_eta", &Tau_eta);
33     fChain->SetBranchAddress("Muon_eta", &Muon_eta);
34     fChain->SetBranchAddress("Electron_eta", &Electron_eta);
35     fChain->SetBranchAddress("Jet_eta", &Jet_eta);
36     fChain->SetBranchAddress("Tau_phi", &Tau_phi);
37     fChain->SetBranchAddress("Muon_phi", &Muon_phi);
38     fChain->SetBranchAddress("Electron_phi", &Electron_phi);
39     fChain->SetBranchAddress("Jet_phi", &Jet_phi);
40
41     // Limiar de pT e eta
42     const float ptCut = 20.0; // Limiar de pT (GeV/c)
```

```

41  const float etaCut = 2.5;  // Limiar de |eta|
42
43  // Criando histogramas
44  TH1F *h_pt_tau = new TH1F("Taupt", "Distribuicao de p_{T} - Tau", 50, 0, 200);
45  TH1F *h_pt_muon = new TH1F("Muonpt", "Distribuicao de p_{T} - Muon", 50, 0, 200);
46  TH1F *h_pt_electron = new TH1F("Electron_pt", "Distribuicao de p_{T} - Eletron", 50,
    0, 200);
47  TH1F *h_pt_jet = new TH1F("Jetpt", "Distribuicao de p_{T} - Jato", 50, 0, 200);
48
49  TH1F *h_eta_tau = new TH1F("Taueta", "Distribuicao de #eta - Tau", 50, -3, 3);
50  TH1F *h_eta_muon = new TH1F("Muoneta", "Distribuicao de #eta - Muon", 50, -3, 3);
51  TH1F *h_eta_electron = new TH1F("Electron_eta", "Distribuicao de #eta - Eletron", 50,
    -3, 3);
52  TH1F *h_eta_jet = new TH1F("Jeteta", "Distribuicao de #eta - Jato", 50, -3, 3);
53
54  TH1F *h_phi_tau = new TH1F("Tauphi", "Distribuicao de #phi - Tau", 50, -TMath::Pi(),
    TMath::Pi());
55  TH1F *h_phi_muon = new TH1F("Muonphi", "Distribuicao de #phi - Muon", 50, -TMath::Pi
    (), TMath::Pi());
56  TH1F *h_phi_electron = new TH1F("Electronphi", "Distribuicao de #phi - Eletron", 50,
    -TMath::Pi(), TMath::Pi());
57  TH1F *h_phi_jet = new TH1F("Jetphi", "Distribuicao de #phi - Jato", 50, -TMath::Pi(),
    TMath::Pi());
58
59  // Criando histograma para massa invariante
60  TH1F *h_mass = new TH1F("in_mass", "Massa Invariante", 50, 0, 200);
61  h_mass->GetXaxis()->SetTitle("Massa Invariante (GeV/c^{2})");
62  h_mass->GetYaxis()->SetTitle("Eventos");
63
64  // Iterando sobre os eventos
65  int nentries = fChain->GetEntries();
66  for (int i = 0; i < nentries; i++) {
67      fChain->GetEntry(i);
68
69  // Preenchendo histogramas de distribui o
70      for (int j = 0; j < nTau; j++) {
71          if (Tau_pt[j] > ptCut && abs(Tau_eta[j]) < etaCut) {
72              h_pt_tau->Fill(Tau_pt[j]);
73              h_eta_tau->Fill(Tau_eta[j]);
74              h_phi_tau->Fill(Tau_phi[j]);
75          }
76      }
77      for (int j = 0; j < nMuon; j++) {
78          if (Muon_pt[j] > ptCut && abs(Muon_eta[j]) < etaCut) {

```

```

79         h_pt_muon->Fill(Muon_pt[j]);
80         h_eta_muon->Fill(Muon_eta[j]);
81         h_phi_muon->Fill(Muon_phi[j]);
82     }
83 }
84 for (int j = 0; j < nElectron; j++) {
85     if (Electron_pt[j] > ptCut && abs(Electron_eta[j]) < etaCut) {
86         h_pt_electron->Fill(Electron_pt[j]);
87         h_eta_electron->Fill(Electron_eta[j]);
88         h_phi_electron->Fill(Electron_phi[j]);
89     }
90 }
91 for (int j = 0; j < nJet; j++) {
92     if (Jet_pt[j] > ptCut && abs(Jet_eta[j]) < etaCut) {
93         h_pt_jet->Fill(Jet_pt[j]);
94         h_eta_jet->Fill(Jet_eta[j]);
95         h_phi_jet->Fill(Jet_phi[j]);
96     }
97 }
98
99 if (nMuon >= 2) { // Garante que h pelo menos dois muons no evento
100     // Preenchendo os quadrivetores para os dois primeiros muons
101     muon1.SetPtEtaPhiM(Muon_pt[0], Muon_eta[0], Muon_phi[0], 0.105); // Massa
102                                     do muon 0.105 GeV
103
104     muon2.SetPtEtaPhiM(Muon_pt[1], Muon_eta[1], Muon_phi[1], 0.105);
105
106     // Calculando a massa invariante
107     double invMass = (muon1 + muon2).M();
108
109     // Preenchendo o histograma de massa invariante
110     h_mass->Fill(invMass);
111 }
112
113 // Criando e salvando graficos de distribuio de pT, eta, phi
114 TCanvas *c1 = new TCanvas("c1", "Distribuicao de pT", 800, 600);
115 c1->Divide(2, 2);
116 c1->cd(1); h_pt_tau->Draw();
117 c1->cd(2); h_pt_muon->Draw();
118 c1->cd(3); h_pt_electron->Draw();
119 c1->cd(4); h_pt_jet->Draw();
120 c1->SaveAs("pt_distribuicao.png");
121
122 TCanvas *c2 = new TCanvas("c2", "Distribuicao de #eta", 800, 600);
123 c2->Divide(2, 2);

```

```

122 c2->cd(1); h_eta_tau->Draw();
123 c2->cd(2); h_eta_muon->Draw();
124 c2->cd(3); h_eta_electron->Draw();
125 c2->cd(4); h_eta_jet->Draw();
126 c2->SaveAs("eta_distribicao.png");
127
128 TCanvas *c3 = new TCanvas("c3", "Distribicao de #phi", 800, 600);
129 c3->Divide(2, 2);
130 c3->cd(1); h_phi_tau->Draw();
131 c3->cd(2); h_phi_muon->Draw();
132 c3->cd(3); h_phi_electron->Draw();
133 c3->cd(4); h_phi_jet->Draw();
134 c3->SaveAs("phi_distribicao.png");
135
136 // Criando e salvando gr fico da massa invariante
137 TCanvas *c4 = new TCanvas("c4", "M", 800, 600);
138 h_mass->Draw();
139 c4->SaveAs("massa_invariante.png");
140
141 // Ajustando o modelo da massa invariante usando uma fun o gaussiana para o sinal
    e uma exponencial para o fundo
142 TF1 *signal = new TF1("signal", "gaus(0)", 0, 200); // Fun o Gaussiana para o
    sinal
143 TF1 *background = new TF1("background", "expo(0)", 0, 200); // Fun o Exponencial
    para o fundo
144
145 // Ajustando a fun o (sinal + fundo)
146 TF1 *fit = new TF1("fit", "gaus(0) + expo(3)", 0, 200);
147 h_mass->Fit(fit, "R");
148
149 // Exibindo os par metros do ajuste (massa do sinal e incertezas)
150 double signalMass = fit->GetParameter(1); // M dia da Gaussiana
151 double signalMassError = fit->GetParError(1); // Erro da m dia da Gaussiana
152 printf("Massa do sinal: %.2f %.2f GeV/c^2\n", signalMass, signalMassError);
153
154 // Salvando a figura do ajuste
155 c4->SaveAs("ajuste_massa_invariante.png");
156
157
158 }

```

Resultados

Esses foram os limiares estabelecidos para pt e eta:

ptCut = 20.0;

etaCut = 2.5;

Aplicando esses cortes, o número de eventos caiu, o que era esperado visto que todos os eventos com $pt > 20$ e $eta > 2.5$ foram descartados.

Estes foram os plots obtidos com este código:

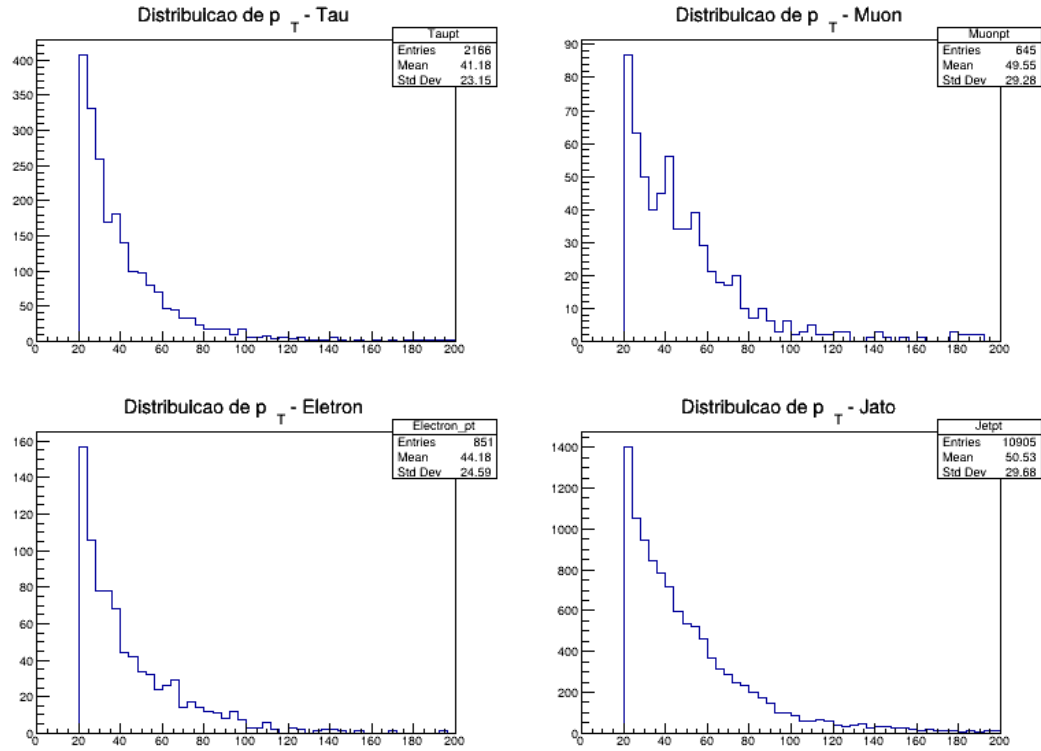


Figure 1: Distribuição de p_T

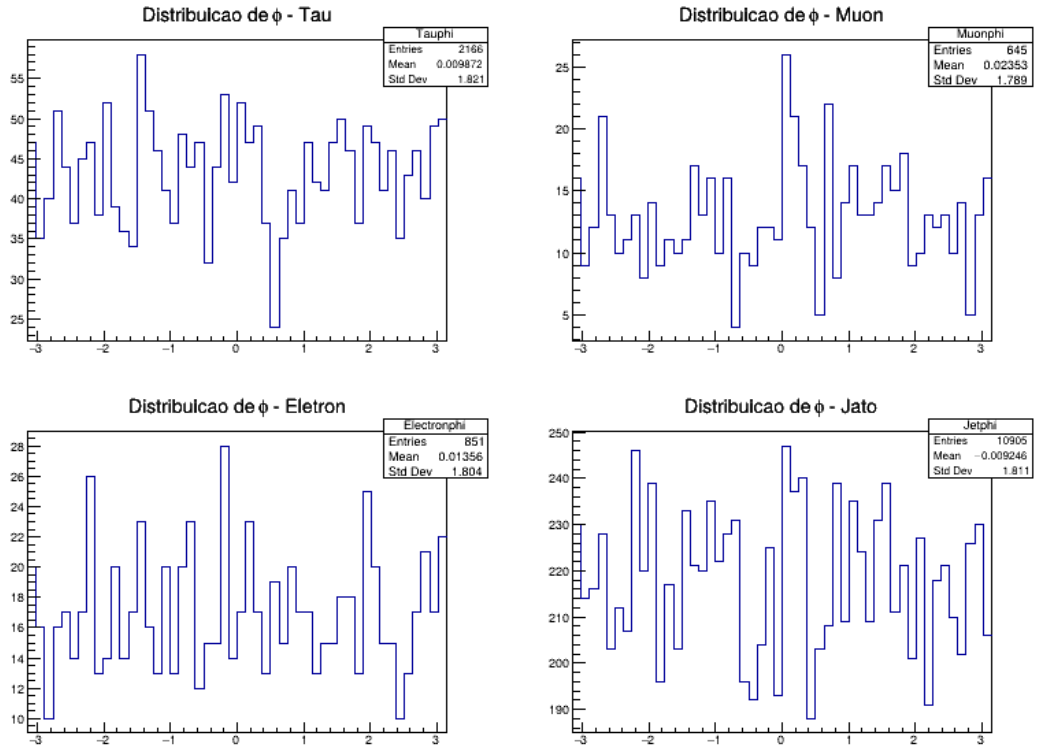


Figure 2: Distribuição de ϕ

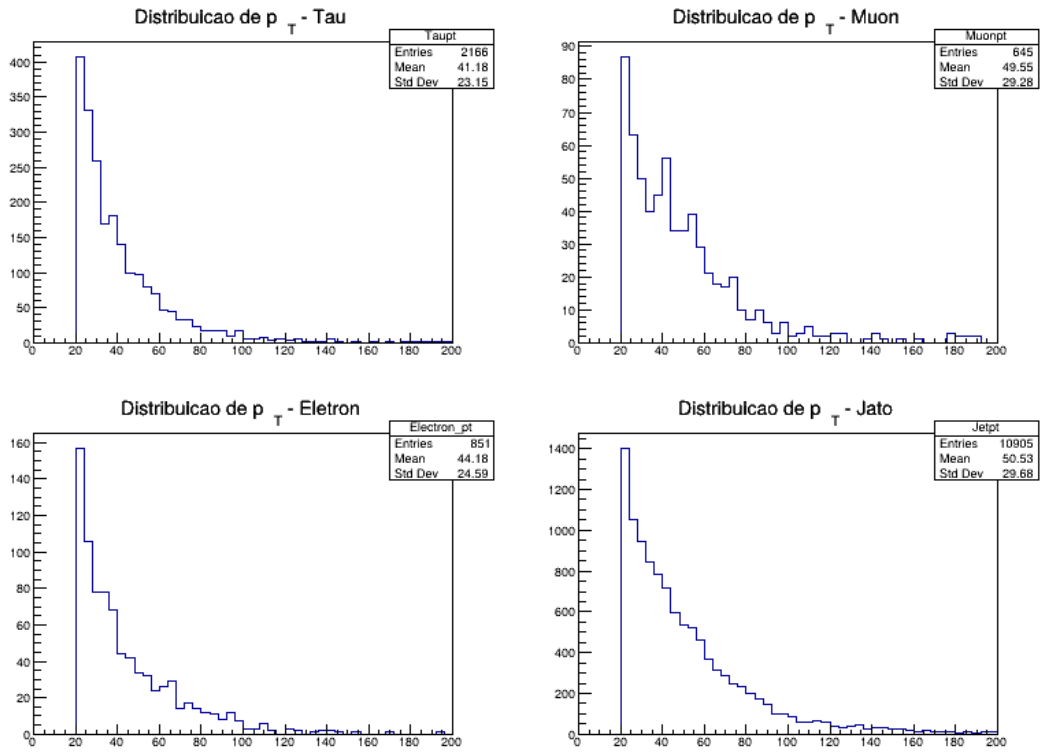


Figure 3: Distribuição de η

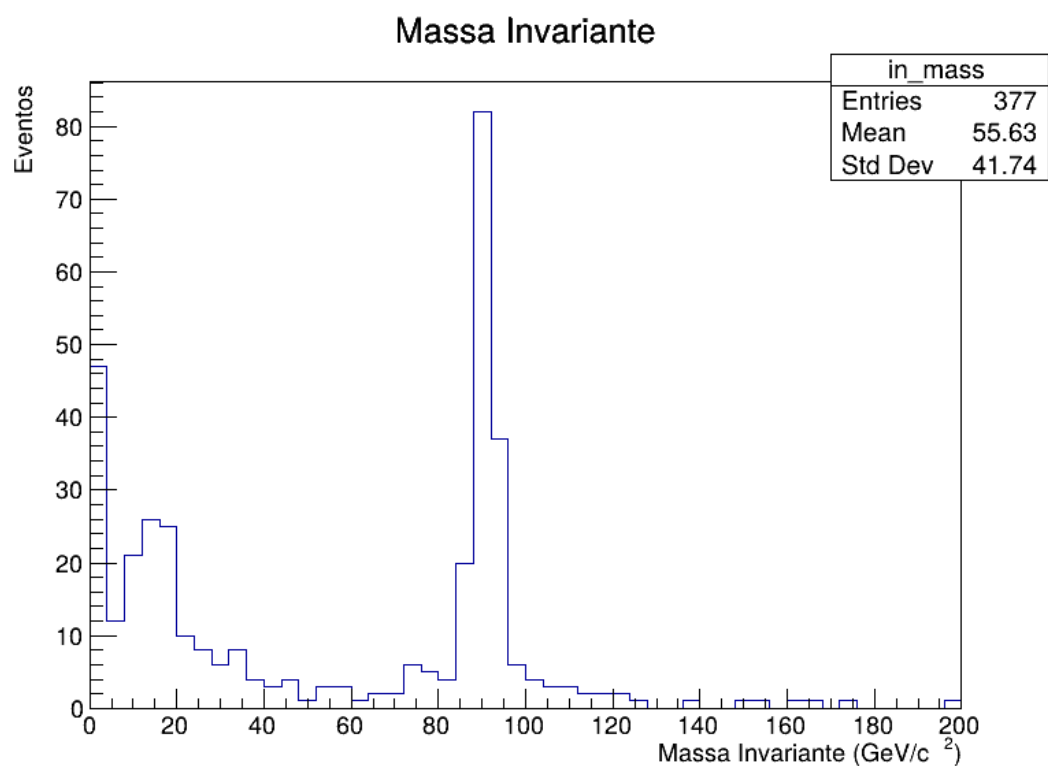


Figure 4: Massa invariante

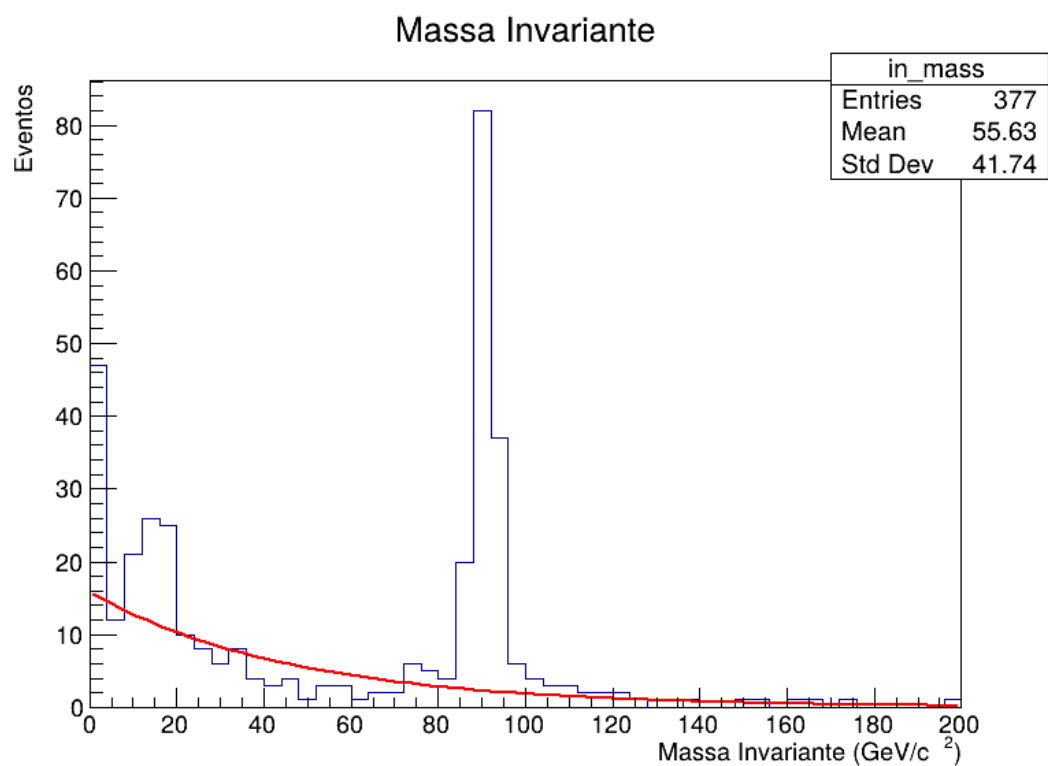


Figure 5: Massa invariante com ajuste